

Predicting English Premier League Match Outcomes with Machine Learning

UE24CS352A Machine Learning - Mini-Project | Sinchana K (PES1UG24AM405), Sanjana Shastry (PES1UG24AM916) | Problem statement no.: ____ | Repo: github.com/<your-username>/epl-match-prediction

1. Problem statement

Given only information available **before** a match (squad ratings), classify an English Premier League game as **home win, draw or home loss**. We study and reproduce Harbola & Lee (Stanford CS229, 2019), comparing several classifiers on feature sets of increasing size and asking how much accuracy is achievable, and why draws are hard to predict.

2. Dataset

3,791 EPL matches (seasons 2009/10-2018/19), scraped from whoscored.com by the paper's authors and released in their public repository. Labels: home win 46.2%, draw 24.8%, home loss 29.0% - so random guessing gives 33.3% and *always predicting a home win* gives 46.2% (our real baseline). Three feature sets: **F1** - home and away overall roster rating (2 features); **F2** - F1 plus GK/DEF/MID/ATT average ratings (10); **F3** - rating, pass %, passes, shots, key passes, blocks and interceptions for every player (280). There are no missing values; features are season-level so the model is treated as time-independent.

3. Approach

- Models:** Gaussian discriminant analysis (GDA), SVM (linear, degree-5 polynomial, RBF), SoftMax regression (linear and quadratic features), one-hidden-layer neural network.
- Protocol (leakage-free):** stratified 80/20 split; standardisation inside a Pipeline; hyper-parameters chosen by 5-fold CV on the training set only; test set scored once. Each tuned model is re-evaluated on 5 further random splits (mean +/- sd), because single-split differences of 1% are within noise.
- Metrics:** accuracy vs the 46.2% baseline, per-class recall (especially draws), macro-F1, confusion matrices.
- Extension:** class-weight balancing to see whether draw recall can be raised and at what cost.

4. Implementation overview

Python 3 / scikit-learn. `data.py` loads the CSVs and builds F1-F3; `models.py` defines the pipelines and grids; `run_experiments.py` tunes, evaluates and saves results; `make_figures.py` produces figures/tables; `demo.py` is the live demo (random unseen matches or custom home/away ratings with class probabilities). SoftMax with quadratic features on F3 (~39k features) was skipped as intractable.

5. Results

Model	F1	F2	F3	F3 train
GDA	56.4	56.6	51.4	64.2
SVM linear	56.3	56.7	55.3	63.1
SVM poly-5	52.1	52.3	47.6	67.8
SVM RBF	55.9	56.7	55.7	77.7
SoftMax linear	56.4	57.0	56.2	59.5
SoftMax quad.	56.4	56.6	-	-
Neural net	55.7	54.8	53.0	67.9

Table 1. Mean test accuracy (%) over 5 random 80/20 splits; last column = training accuracy on the main split. Baselines: random 33.3, home-win 46.2.

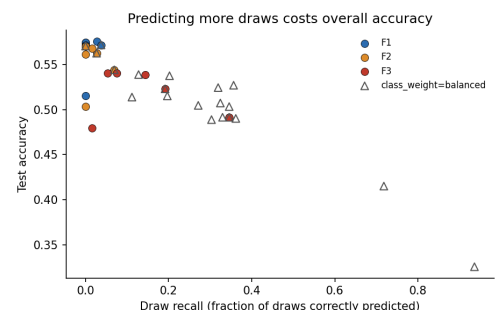


Figure 1. Draw recall vs test accuracy, all runs (triangles: class-weight balanced).

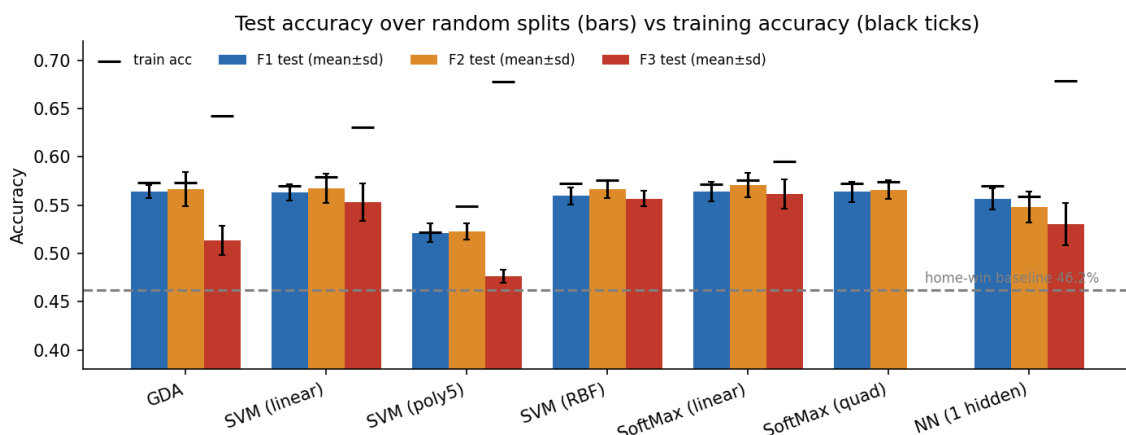


Figure 2. Test accuracy (bars, mean +/- sd over splits) vs training accuracy (black ticks).

6. Conclusions

- Squad ratings alone give **~56-57% accuracy**, about 10-11 points above the home-win baseline (46.2%); the best model by CV was linear SVM on F2 (57.7% CV, 57.0% held-out), and SoftMax (linear) on F2 the best mean over splits (57.0%). Differences among the good models (~1%) are within split-to-split noise (sd 0.7-1.8%), so a *simple* model is as good as a complex one.
- **More features hurt:** on F3 training accuracy rises to 63-78% while test accuracy falls to 47-56% (overfitting; F3 has 280 features for ~3,000 training rows).
- **Draws are the bottleneck:** on F1/F2 well-generalising models predict draws almost never (recall $\leq 7\%$) because draws sit between wins and losses in rating space. F3 models predict more draws (recall up to 35%) but through overfitting, lowering accuracy. Class balancing also raises draw recall (to 20-36%) but drops accuracy to ~50-54%.
- **Reproduction note:** the paper's 58.89% (poly-5 SVM, F1) came from a single split; under our CV-tuned, standardised pipeline that model reached only ~52% and was unstable. Its qualitative findings (~57% from ratings, overfitting with larger feature sets, hard draws) are reproduced; that exact figure is not.
- **Future work:** add recent-form/head-to-head features, use ordinal or two-stage (win-vs-not, then draw) models, probability calibration, and gradient boosting.

Reference: V. Harbola, K. Lee, *Gaining a Statistical Edge in Soccer Prediction using Machine Learning*, CS229 Final Project, Stanford, 2019 (data and repo: github.com/varunharbola/EPL_match_prediction).